

Python List Revision Chart

This revision sheet covers Python list properties, mutability, built-in functions, methods, operations, and slicing with start, stop, and step.[cite:27][cite:83][cite:69]

List chart

Section	Number	Topic	Explanation	Example	Output
Properties	1	Syntax	List is written with square brackets []. [cite:83] [cite:85]	a = [1, 2, 3]	[1, 2, 3]
Properties	2	Empty list	Empty list can be written as [] or list(). [cite:83] [cite:87]	a = []	[]
Properties	3	Mutability	List is mutable, so elements can be added, changed, or removed after creation. [cite:83] [cite:86]	a = [1, 2] then a.append(3)	[1, 2, 3]
Properties	4	Ordered	List keeps elements in a fixed order. [cite:83] [cite:86]	a = [10, 20, 30]	Order is fixed
Properties	5	Indexing allowed	List supports indexing and starts from 0. [cite:83] [cite:85]	a = [10, 20, 30] print(a[0])	10
Properties	6	Slicing allowed	List supports slicing with start, stop, and step.[cite:84] [cite:69]	a = [0,1,2,3,4,5] print(a[1:5:2])	[1, 3]
Properties	7	Duplicate values	List allows duplicate values.[cite:83] [cite:87]	a = [1, 2, 2, 3]	[1, 2, 2, 3]

Section	Number	Topic	Explanation	Example	Output
Properties	8	Mixed data types	List can store different data types together. [cite:86] [cite:87]	a = [1, "hi", 3.5]	[1, 'hi', 3.5]
Properties	9	Membership testing	List supports <code>in</code> and <code>not in</code> for checking elements. [cite:84] [cite:85]	print(2 in [1, 2, 3])	True
Built-in Function	1	len()	Returns total number of elements. [cite:83] [cite:84]	print(len([1, 2, 3]))	3
Built-in Function	2	min()	Returns smallest element. [cite:27]	print(min([3, 1, 2]))	1
Built-in Function	3	max()	Returns largest element. [cite:27]	print(max([3, 1, 2]))	3
Built-in Function	4	sum()	Returns sum of numeric elements. [cite:27]	print(sum([1, 2, 3]))	6
Built-in Function	5	sorted()	Returns a new sorted list. [cite:27] [cite:85]	print(sorted([3, 1, 2]))	[1, 2, 3]
Built-in Function	6	list()	Creates a list from an iterable. [cite:85] [cite:87]	print(list((1,2,3)))	[1, 2, 3]
Built-in Function	7	in	Checks whether an element exists in the list. [cite:84] [cite:85]	print(2 in [1, 2, 3])	True
Built-in Function	8	not in	Checks whether an element does not exist in the list.[cite:84] [cite:85]	print(5 not in [1, 2, 3])	True

Section	Number	Topic	Explanation	Example	Output
Method	1	<code>append()</code>	Adds one element at the end of the list. [cite:27] [cite:85]	<code>a = [1, 2] a.append(3) print(a)</code>	<code>[1, 2, 3]</code>
Method	2	<code>extend()</code>	Adds all elements from another iterable. [cite:27]	<code>a = [1, 2] a.extend([3,4]) print(a)</code>	<code>[1, 2, 3, 4]</code>
Method	3	<code>insert()</code>	Inserts an element at a given index. [cite:27]	<code>a = [1, 3] a.insert(1, 2) print(a)</code>	<code>[1, 2, 3]</code>
Method	4	<code>remove()</code>	Removes the first matching value.[cite:27] [cite:85]	<code>a = [1, 2, 2, 3] a.remove(2) print(a)</code>	<code>[1, 2, 3]</code>
Method	5	<code>pop()</code>	Removes and returns the element at a given index, or the last item by default.[cite:27] [cite:85]	<code>a = [10, 20, 30] print(a.pop())</code>	<code>30</code>
Method	6	<code>clear()</code>	Removes all elements from the list.[cite:27]	<code>a = [1, 2] a.clear() print(a)</code>	<code>[]</code>
Method	7	<code>index()</code>	Returns the index of the first occurrence of a value. [cite:27] [cite:85]	<code>print([10,20,30].index(20))</code>	<code>1</code>
Method	8	<code>count()</code>	Returns how many times a value appears. [cite:27]	<code>print([1,2,2,3].count(2))</code>	<code>2</code>
Method	9	<code>sort()</code>	Sorts the list in place.[cite:27] [cite:85]	<code>a = [3,1,2] a.sort() print(a)</code>	<code>[1, 2, 3]</code>
Method	10	<code>reverse()</code>	Reverses the list in place. [cite:27] [cite:85]	<code>a = [1,2,3] a.reverse() print(a)</code>	<code>[3, 2, 1]</code>
Method	11	<code>copy()</code>	Creates a shallow copy of the list.[cite:27]	<code>a = [1,2] b = a.copy() print(b)</code>	<code>[1, 2]</code>

Section	Number	Topic	Explanation	Example	Output
Operation	1	Concatenation +	Joins two lists into a new list. [cite:84] [cite:85]	<code>print([1,2] + [3,4])</code>	<code>[1, 2, 3, 4]</code>
Operation	2	Repetition *	Repeats the list n times. [cite:85]	<code>print([1,2] * 3)</code>	<code>[1, 2, 1, 2, 1, 2]</code>
Operation	3	Membership in	Checks whether an element is present. [cite:84] [cite:85]	<code>print(3 in [1,2,3])</code>	True
Operation	4	Membership not in	Checks whether an element is absent. [cite:84] [cite:85]	<code>print(7 not in [1,2,3])</code>	True
Operation	5	Delete using del	Deletes an element by index or slice. [cite:84]	<code>a = [1,2,3]</code> <code>del a[1]</code> <code>print(a)</code>	<code>[1, 3]</code>

Start, stop, step in slicing

List supports slicing with the syntax `a[start:stop:step]`. [cite:69] In this syntax, `start` is the beginning index, `stop` is the ending limit, and `step` is the interval between selected elements. [cite:69]

Example:

```
a = [0, 1, 2, 3, 4, 5]
print(a[1:5:2])
```

Output:

```
[1, 3]
```

Note

Start, stop, step in slicing: List supports slicing with the syntax `a[start:stop:step]`. In this syntax, `start` is the beginning index, `stop` is the ending limit, and `step` is the interval between selected elements. [cite:69]

Mutability note

List is mutable because values can be changed directly and methods like `append()`, `extend()`, `insert()`, `remove()`, `pop()`, `clear()`, `sort()`, and `reverse()` modify the same list object.[cite:27]
[cite:83][cite:86]